

КУРС «ОБЪЕКТНО-ОРИЕНТИРОВАННОЕ ПРОГРАММИРОВАНИЕ» · 1 СЕМЕСТР

02

Лекция 2
Операторы, ввод-вывод, управляющие конструкции

Арифметика и побитовые операции · приоритет и инкремент · составные операторы · ввод-вывод · if/switch · циклы · преобразования типов

Вспоминаем, прошлую лекцию

```
A { int value = 1; }
namesnamespacepace B { int value = 2; }

using namespace A;
using namespace B;

int getSize() { return 5; }

int main() {
    int x = value;
    int a{3.14};
    int b = {2.5};
    unsigned c{-1};

    const int d;
    d = 10;

    int e;
    int f = e;

    const int size = getSize();
    int arr[size];

    return 0;
}
```



ПЛАН ЛЕКЦИИ

Одиннадцать глав сегодняшней лекции

01 Арифметические операторы

02 Побитовые операторы

03 Логические операторы и сравнения

04 Приоритет и инкремент

05 Составные операторы · numeric_limits

06 Математические функции <cmath>

07 Потоковый ввод-вывод

08 Условные конструкции

09 Циклы

10 break, continue, запятая

11 Преобразования типов

Сквозная идея: операторы — язык выражений; управляющие конструкции — язык потока. Вместе они строят любую программу.



Целочисленное деление усекает

```
int    a  =  17, b    =  5;
a +    b     // 22
a -    b     // 12
a *    b     // 85
a /    b     // 3      — целочисленное      деление!
a %    b     // 2      — остаток

double    c =    17.0, d = 5.0;
c / d         //   3.4

// 5 + 2.0 → double (usual conversions)
// INT_MAX + 1 → UB для signed!
```

- **17 / 5 == 3** — усечение к нулю; %— только для целых
- деление/остаток на **ноль** — UB
- переполнение **signed** — UB; unsigned— wrap-around по модулю 2^n
- перед операцией — продвижения и usual arithmetic conversions

ПРИМЕР

Смотрим пример: 01_arithmetic



https://github.com/DVDemon/OOP_2027_Part_01/tree/main/lecture_02_operators_io/01_arithmetic

Работа с отдельными битами

```
unsigned a = 0b1100, b = 0b1010;
a & b       // 1000       AND
a | b       // 1110       OR a
^ b         // 0110       XOR
~a          // ...0011          NOT
a << 1      // 00011000         сдвиг влево
a >> 1      // 0110       сдвиг вправо

// Битовые флаги
constexpr unsigned READ=0b001, WRITE=0b010;
unsigned p = READ | WRITE;                    // установить
bool ok = (p & READ) !=0;                     // проверить
p &= ~WRITE;                                   // сбросить
```

- работают с целыми типами — предпочитайте unsigned
- сдвиг влево signed с переполнением — UB;

  вправо отрицательного — implementation-defined
- битовые флаги: |=· &· &= ~ std::bitset—
- удобная печать битов

ПРИМЕР

Смотрим пример: 02_bitwise



https://github.com/DVDemon/OOP_2027_Part_01/tree/main/lecture_02_operators_io/02_bitwise

Короткая схема вычислений

```
bool a = true, b = false;
a && b        // false
a || b        // true
!a            // false

int x = 10, y = 20;
x == y        // false;          x < y       // true

// short-circuit: правый операнд
// вычисляется, только если нужен
int* ptr = nullptr;
if (ptr != nullptr && *ptr > 0) {
        // безопасно при nullptr
}
```

- **short-circuit** в &&и ||— защита от опасных вычислений
- возвращают bool
- doubleчерез ==нельзя — сравнивайте с epsilon

ПРИМЕР

Смотрим пример: 03_logical_comparison



https://github.com/DVDemon/OOP_2027_Part_01/tree/main/lecture_02_operators_io/03_logical_comparison

Префиксный и постфиксный инкремент

```
int a = 5; int b =
++a; int c = a++;       // a=6, b=6 — префиксный
                        // c=6, a=7 — постфиксный

// Приоритет: постфиксные > унарные
//       2: a++        a--
//       3: ++a        --a      !     ~     *     &
//       ...
//       16: =       +=     -= ...

// при сомнении — скобки!
```

- постфиксный ++выше префиксного по приоритету
- для встроенных типов разницы нет; для итераторов — предпочитайте ++i
- правоассоциативны унарные и присваивание
- **читаемость важнее краткости** — ставьте скобки

ПРИМЕР

Смотрим пример: 04_precedence_increment



https://github.com/DVDemon/OOP_2027_Part_01/tree/main/lecture_02_operators_io/04_precedence_increment

x op= y — один раз вычисляем x

```
int x = 10;
x += 5;          // 15
x -= 3;          // 12
x *= 2;          // 24
x /= 4;          // 6
x %= 4;          // 2
x <<= 2;         // 8

// Типобезопасные константы
std::numeric_limits<int>::max()
std::numeric_limits<double>::epsilon()
std::numeric_limits<double>::quiet_NaN()
// NaN != NaN — проверяйте std::isnan
```

- x op= y x = x op y, но левый операнд — один раз
- нет составных форм для &&, ||, запятой
- numeric_limitsвместо макросов INT_MAX/DBL_MAX min()y float
- — наименьшее нормализованное; lowest()
  — наименьшее конечное

ПРИМЕР

Смотрим пример: 05_compound_numeric_limits



https://github.com/DVDemon/OOP_2027_Part_01/tree/main/lecture_02_operators_io/05_compound_numeric_limits

Функции из <cmath>

```
std::abs(-5)              // 5
std::sqrt(144.0)          // 12
std::pow(2.0, 10) // 1024
std::ceil(3.2)            // 4
std::floor(3.8)           // 3
std::round(3.5)           // 4

std::sin(M_PI / 2)            // 1
std::log(M_E)                 // 1.0
std::log10(1000.0)            // 3.0
std::log2(1024.0)             // 10.0
```

- работают с double; углы — в радианах
- в C++20 —std::numbers::pi
- std::absперегружен для целых и double
- pow(x, 2)может быть медленнее x * x

ПРИМЕР

Смотрим пример: 06_cmath



https://github.com/DVDemon/OOP_2027_Part_01/tree/main/lecture_02_operators_io/06_cmath

std::cout, манипуляторы, std::cin

```
std::cout << std::fixed
          << std::setprecision(2) << pi;          // 3.14 std::cout
<< std::setw(10) << std::right << 42; std::cout << std::hex << std::showbase <<
255; // 0xff std::cout << std::boolalpha << true;          // true

// Ввод
int age;
if (std::cin >> age) { /* ok */ }
else { std::cin.clear(); std::cin.ignore(); }

std::cin.ignore();                    // после >>
std::string line; std::getline(std::cin, line);
```

- >>— до пробела; getline— до конца строки
- результат ввода — условие: if (std::cin >> x) setw
- действует на одно значение
- предпочитайте '\n'вместо std::endl(сброс буфера)

ПРИМЕР

Смотрим пример: 07_stream_io



https://github.com/DVDemon/OOP_2027_Part_01/tree/main/lecture_02_operators_io/07_stream_io

if, init-if (C++17), switch

```
if (score >= 90) { ... }
else if (score >= 75) { ... }
else { ... }

// C++17 init-if: it живёт в рамках if
if (auto it = grades.find("Иванов"); it != grades.end()) {
        std::cout << it->second;
}

switch (color) {
        case Color::Red:                ... break; case
        Color::Green: ... break;
        // [[fallthrough]] — намеренное проваливание
}
```

- первое истинное условие — порядок важен
- init-ifограничивает область видимости переменной
- забытый break— проваливание; намеренное — [[fallthrough]]

ПРИМЕР

Смотрим пример: 08_conditionals



https://github.com/DVDemon/OOP_2027_Part_01/tree/main/lecture_02_operators_io/08_conditionals

Выход из цикла и следующий шаг

```
for (int i = 0; i < 100; ++i) {
        if (i * i > 50) break;                  // выход
}

for (int i = 0; i < 20; ++i) {
        if (i % 3 == 0) continue; // к шагу цикла
        std::cout << i << " ";
}

// запятая: результат — правый операнд
int a = (1, 2, 3);                  // a = 3
for (int i = 0, j = 10; i < j; ++i, --j) { }
```

- break— ближайший цикл/switch; continue— следующая итерация
- глубокие вложенные циклы с break— выносите логику в функцию
- запятая вне forухудшает читаемость

ПРИМЕР

Смотрим пример: 09_loops



https://github.com/DVDemon/OOP_2027_Part_01/tree/main/lecture_02_operators_io/09_loops

ПРИМЕР

Смотрим пример: 10_break_continue_comma



https://github.com/DVDemon/OOP_2027_Part_01/tree/main/lecture_02_operators_io/10_break_continue_comma

Неявные и явные преобразования

```
// Неявные int i
= 42; int j = d;        double d = i;                // расширение
                                                       // сужение + warning

short s = 10;
auto r = s + 1;                    // promotion: short → int

int a = 5;              double b = 2.5;
auto c = a + b;                    // usual: int → double

// Явные
int t = static_cast<int>(3.14159);                          // 3
// C-style (int)pi — слишком мощный, опасен
```

- продвижения и usual arithmetic conversions «выравнивают» типы
- сужающие теряют данные — {}-инициализация их запрещает
- именованные static_cast/const_cast/reinterpret_cast/dynamic_cast

ПРИМЕР

Смотрим пример: 11_type_conversions



https://github.com/DVDemon/OOP_2027_Part_01/tree/main/lecture_02_operators_io/11_type_conversions

ИТОГИ

Ключевые концепции лекции

- **Целочисленное деление** отбрасывает дробную часть: `7 / 2 == 3`
- **Short-circuit** в `&&`/`||` — правый операнд может не вычисляться
- **`++i`** предпочтительнее `i++` для итераторов
- **`numeric_limits`** вместо макросов `INT_MAX`
- **Range-for** с `const auto&` — перебор без копий
- **init-if** (C++17) сужает область видимости
- **Неявные преобразования** могут терять данные — защищайтесь `{}`

> Операторы и управляющие конструкции — база любой программы; примеры к каждой главе — на GitHub.

ПРАКТИКА

Что чаще всего идёт не так

```
// 1. double через ==
double a = 0.1 + 0.2;
if (a == 0.3) { }                  // может не сработать
// правильно: |a - 0.3| < eps

// 2. Забытый break в switch
case 1: cout << "one";
                // проваливаемся в case 2!

// 3. = вместо ==
if (x = 0) { }                     // присваивание!
```

```
// 4. Переполнение signed
int x = numeric_limits<int>::max(); x += 1;
                    // UB!

// 5. cin >> и getline cin >> n;
getline(cin, line);                          // '\n' в буфере
                                             // пустая строка!
// решение: cin.ignore() между ними
```

ПРАКТИКА

Задание: «Статистический калькулятор»

- читать целые с std::cindo 0(через do-while)
- посчитать: количество, сумму, среднее (2 знака), min/max, чётные/нечётные
- вывести отформатированную таблицу: setw, left, fixed, setprecision
- сразу 0→ «Пустая последовательность» (if)
- min/max инициализировать через numeric_limits<int>::min()/max()

```
// Флаги компиляции
-std=c++20 -Wall -Wextra

// Никаких глобальных переменных
// structured bindings (C++17) — бонус

Пример ввода:
5 3 8 1 0
→ 4 числа, сумма 17,
   среднее 4.25, min 1, max 8,
   чётных 2, нечётных 2
```

> Вопросы для самопроверки — в конце конспекта лекции.